

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline
```

```
In [ ]: df = pd.read_csv("BostonHousing.csv")
df = df.dropna()
df
```

```
In [ ]: df.head()
```

```
In [ ]: df.columns
```

```
In [ ]: x = df[['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE',
               'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']]
x.head()
```

```
In [ ]: y = df[['MEDV']]
y.head()
```

```
In [ ]: x.shape, y.shape
```

```
In [ ]: x.info()
```

```
In [ ]: x.describe()
```

```
In [ ]: y.info()
```

```
In [ ]: y.describe()
```

```
In [ ]: x.isnull().sum()
```

```
In [ ]: y.isnull().sum()
```

```
In [ ]: df = x
df["target"] = y
df.head()
```

```
In [ ]: plt.figure(figsize=(15,10))
sns.heatmap(df.corr(), annot=True)
plt.show()
```

```
In [ ]: df = df[['RM', 'LSTAT', 'target']]
```

```
In [ ]: sns.pairplot(df)
```

```
plt.show()
```

```
In [ ]: x = df[['RM', 'LSTAT']]  
y = df['target']
```

```
In [ ]: scaler = StandardScaler()
```

```
In [ ]: x = scaler.fit_transform(x)
```

```
In [ ]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3,
```

```
In [ ]: x_train
```

```
In [ ]: model = LinearRegression(n_jobs=-1)
```

```
In [ ]: model.fit(x_train, y_train)
```

```
In [ ]: y_pred = model.predict(x_test)
```

```
In [ ]: y_pred
```

```
In [ ]: mean_squared_error(y_test, y_pred)
```

```
In [ ]: mean_absolute_error(y_test, y_pred)
```

```
In [ ]: sns.regplot(x=y_test, y= y_pred, color='red')  
plt.show()
```

```
In [ ]: plt.scatter(y_test, y_pred)  
plt.xlabel('Y Test')  
plt.ylabel('Predicted Y')  
plt.show()
```

```
In [ ]: mean_absolute_error(y_test, y_pred)
```

```
In [ ]: df1 = pd.DataFrame({'Actual':y_test, 'Predicted':y_pred, 'Variance':y_test - y_pred})  
df1.head()
```

```
In [ ]: df.head(15)
```

```
In [ ]:
```